AI Lab assignment 10

Tejas Patil
U24AI061

Q1)

```cpp
#include <bits/stdc++.h>
using namespace std;

double dist_sq(vector<double> a, vector<double> b) {
    return (a[0]-b[0])*(a[0]-b[0]) + (a[1]-b[1])*(a[1]-b[1]);
}

int main() {
    vector<vector<double>> data;
    ifstream file("cities.csv");

    double x, y;
    while (file >> x >> y) {
        data.push_back({x, y});
    }

    int k = 3;
    vector<vector<double>> centers(k);

    // Initial centers
    for (int i = 0; i < k; i++)
        centers[i] = data[i];

    double alpha = 0.1;

    for (int iter = 0; iter < 20; iter++) {
        vector<vector<vector<double>>> clusters(k);

        // Assign cities
        for (auto &p : data) {
            int idx = 0;
            double best = 1e18;
```

```
            for (int i = 0; i < k; i++) {
                double d = dist_sq(p, centers[i]);
                if (d < best) {
                    best = d;
                    idx = i;
                }
            }
            clusters[idx].push_back(p);
        }

        // Update centers (Gradient Descent)
        for (int i = 0; i < k; i++) {
            if (clusters[i].empty()) continue;

            double gx = 0, gy = 0;

            for (auto &p : clusters[i]) {
                gx += (centers[i][0] - p[0]);
                gy += (centers[i][1] - p[1]);
            }

            gx *= 2;
            gy *= 2;

            centers[i][0] -= alpha * gx / clusters[i].size();
            centers[i][1] -= alpha * gy / clusters[i].size();
        }
    }

    // Output
    cout << "Gradient Descent Centers:\n";
    for (auto &c : centers)
        cout << c[0] << " " << c[1] << endl;

    // Compute SSD
    double ssd = 0;
    for (auto &p : data) {
        double best = 1e18;
        for (int i = 0; i < k; i++) {
            best = min(best, dist_sq(p, centers[i]));
```

```
        }
        ssd += best;
    }

    cout << "SSD (GD): " << ssd << endl;

    return 0;
}
```

Q1B)

```
#include <bits/stdc++.h>
using namespace std;

double dist_sq(vector<double> a, vector<double> b) {
    return (a[0]-b[0])*(a[0]-b[0]) + (a[1]-b[1])*(a[1]-b[1]);
}

int main() {
    vector<vector<double>> data;
    ifstream file("cities.csv");

    double x, y;
    while (file >> x >> y) {
        data.push_back({x, y});
    }

    int k = 3;
    vector<vector<double>> centers(k);

    // Initial centers
    for (int i = 0; i < k; i++)
        centers[i] = data[i];

    for (int iter = 0; iter < 20; iter++) {
        vector<vector<vector<double>>> clusters(k);

        // Assign cities
        for (auto &p : data) {
```

```
            int idx = 0;
            double best = 1e18;

            for (int i = 0; i < k; i++) {
                double d = dist_sq(p, centers[i]);
                if (d < best) {
                    best = d;
                    idx = i;
                }
            }
            clusters[idx].push_back(p);
        }

        // Update centers (Newton Method)
        for (int i = 0; i < k; i++) {
            if (clusters[i].empty()) continue;

            double gx = 0, gy = 0;

            for (auto &p : clusters[i]) {
                gx += (centers[i][0] - p[0]);
                gy += (centers[i][1] - p[1]);
            }

            gx *= 2;
            gy *= 2;

            // Divide by Hessian (which is 2)
            centers[i][0] -= gx / (2 * clusters[i].size());
            centers[i][1] -= gy / (2 * clusters[i].size());
        }
    }

    // Output
    cout << "Newton Method Centers:\n";
    for (auto &c : centers)
        cout << c[0] << " " << c[1] << endl;

    // Compute SSD
    double ssd = 0;
```

```
    for (auto &p : data) {
        double best = 1e18;
        for (int i = 0; i < k; i++) {
            best = min(best, dist_sq(p, centers[i]));
        }
        ssd += best;
    }

    cout << "SSD (Newton): " << ssd << endl;

    return 0;
}
```

Q2)

```
#include <bits/stdc++.h>
using namespace std;

// State = (location, A status, B status)
struct State {
    string loc, A, B;

    bool operator==(const State &other) const {
        return loc == other.loc && A == other.A && B == other.B;
    }
};

// For using State in set/vector search
bool contains(vector<State> &path, State s) {
    for (auto &p : path) {
        if (p == s) return true;
    }
    return false;
}

// Goal test
bool is_goal(State s) {
    return s.A == "Clean" && s.B == "Clean";
}
```

```
// Possible actions
vector<string> actions() {
    return {"Suck", "Left", "Right"};
}

// Result function (non-deterministic)
vector<State> results(State s, string action) {
    vector<State> res;

    if (action == "Suck") {
        if (s.loc == "A") {
            if (s.A == "Dirty") {
                res.push_back({"A", "Clean", s.B});
                res.push_back({"A", "Clean", "Clean"});
            } else {
                res.push_back({"A", "Clean", s.B});
                res.push_back({"A", "Dirty", s.B});
            }
        } else {
            if (s.B == "Dirty") {
                res.push_back({"B", s.A, "Clean"});
                res.push_back({"B", "Clean", "Clean"});
            } else {
                res.push_back({"B", s.A, "Clean"});
                res.push_back({"B", s.A, "Dirty"});
            }
        }
    }
    else if (action == "Left") {
        res.push_back({"A", s.A, s.B});
    }
    else if (action == "Right") {
        res.push_back({"B", s.A, s.B});
    }

    return res;
}

// AND-OR Search
```

```
// Returns pointer to plan (vector<string>) or nullptr
vector<string>* and_or_search(State state, vector<State> path) {
    if (is_goal(state)) {
        return new vector<string>(); // empty plan
    }

    if (contains(path, state)) {
        return nullptr;
    }

    for (string action : actions()) {
        vector<State> result_states = results(state, action);

        vector<string> plan;
        bool success = true;

        for (auto &s : result_states) {
            vector<State> new_path = path;
            new_path.push_back(state);

            vector<string>* subplan = and_or_search(s, new_path);

            if (subplan == nullptr) {
                success = false;
                break;
            }

            // Append subplan
            plan.insert(plan.end(), subplan->begin(), subplan->end());
        }

        if (success) {
            vector<string>* final_plan = new vector<string>();
            final_plan->push_back(action);
            final_plan->insert(final_plan->end(), plan.begin(),
plan.end());
            return final_plan;
        }
    }
```

```
    return nullptr;
}

int main() {
    State start = {"B", "Dirty", "Dirty"};

    vector<State> path;

    vector<string>* plan = and_or_search(start, path);

    cout << "Plan:\n";

    if (plan == nullptr) {
        cout << "No solution\n";
    } else {
        for (auto &step : *plan) {
            cout << step << " ";
        }
        cout << endl;
    }

    return 0;
}
```